

Web Data Scraping Project Report

Hockey Standings Web Scraper

Student Name: Amidu Dabor

Track: Data and AI

Date: 18 May, 2025

Link to notebook: <https://drive.google.com/file/d/1Nx9rM71ajQegcxUM5O28wuXrFlwis7WP/view?usp=sharing>

1. Introduction

This report documents the development and implementation of a comprehensive web scraping solution designed to extract hockey team standings data from a live website. The project demonstrates the practical application of web scraping techniques using Python libraries to gather, process, and analyse structured data from online sources.

1.1 Project Objectives

The main objectives of this project were to:

1. Develop a robust, object-oriented implementation of a web scraper using Python
2. Extract structured hockey standings data from a target website
3. Process and clean the extracted data for analysis
4. Conduct basic data analysis on the extracted information

5. Visualize key insights from the data
6. Export the processed data to a CSV file for further use

1.2 Technologies Used

The following technologies and libraries were utilized in this project:

```
Python 3.11.8: Core programming language
Requests: HTTP library for making web requests
BeautifulSoup4: HTML parsing library
Pandas: Data manipulation and analysis library
NumPy: Numerical computing library
Matplotlib & Seaborn: Data visualization libraries
Tabulate: Pretty-printing tabular data
Logging: Python's built-in logging module for tracking execution
```

2. Implementation Approach

2.1 Object-Oriented Design

I implemented the project using an object-oriented approach, creating a Hockey Scraper class that encapsulates all the necessary functionality. This design promotes code reusability, maintainability, and a clear separation of concerns.

The Hockey Scraper class follows a modular design with methods for each step in the web scraping process:

```
class HockeyScraper:
```

```
"""
A class for scraping hockey standings data from websites.
"""

def __init__(self, url, output_dir="./output", log_level=logging.INFO, verbose=True):
    # Initialize configuration and state variables

def fetch_page(self, max_retries=3, delay=2):
    # Fetch HTML content with retry logic

def inspect_html(self, max_length=500):
    # Analyze HTML structure to guide extraction

def extract_table_data(self, table_class='table'):
    # Extract structured data from HTML tables

def analyze_data(self):
    # Perform statistical analysis on the extracted data

def visualize_data(self, save_path=None):
    # Create visualizations of key statistics

def save_to_csv(self, filename=None):
    # Export processed data to CSV format

def run_complete_workflow(self, save_visualizations=True):
    # Orchestrate the entire scraping process
```

This modular structure allows each component to be developed, tested, and maintained independently, while also enabling a smooth, integrated workflow.

2.2 Robust Error Handling

The implementation incorporates comprehensive error handling throughout each step of the process:

```
def fetch_page(self, max_retries=3, delay=2):
    """Fetch the webpage content from the URL."""
    self.logger.info(f"Fetching page content from {self.url}")

    for attempt in range(max_retries):
        try:
            response = requests.get(self.url, headers=self.headers, timeout=10)
            response.raise_for_status()

            # Process successful response
            # ...

            return True

        except requests.exceptions.RequestException as e:
            self.logger.warning(f"Attempt {attempt + 1}/{max_retries} failed: {str(e)}")

            if attempt < max_retries - 1:
                # Implement exponential backoff for retries
                wait_time = delay * (2 ** attempt)
                self.logger.info(f"Waiting {wait_time} seconds before retrying...")
                time.sleep(wait_time)
            else:
                self.logger.error(f"Failed to fetch page after {max_retries} attempts")
                return False

    return False
```

This approach ensures that the scraper can handle various error conditions, including network issues, server errors, and unexpected HTML structures, without crashing or producing inconsistent results.

2.3 Adaptive Extraction Strategy

The data extraction process is designed to be flexible and adaptable to different HTML structures:

```

def extract_table_data(self, table_class='table'):
    """Extract data from the HTML table on the page."""
    try:
        # Find the table by class name
        table = self.soup.find('table', class_=table_class)
        if not table:
            # Fallback to any table if no table with specified class is found
            table = self.soup.find('table')
            if table:
                self.logger.warning(f"No table found with class '{table_class}', using first table found instead")
            else:
                return False

        # Extract headers using different strategies
        header_elements = table.find_all('th')
        if header_elements:
            self.table_headers = [header.text.strip() for header in header_elements]
            row_elements = table.find_all('tr')[1:] # Skip header row
        else:
            # Try using the first row as headers
            first_row = table.find('tr')
            if first_row:
                self.table_headers = [cell.text.strip() for cell in first_row.find_all(['td', 'th'])]
                row_elements = table.find_all('tr')[1:] # Skip first row as it's now headers
            else:
                return False

        # Extract and normalize row data
        # ...

        return True

    except Exception as e:
        self.logger.error(f"Failed to extract table data: {str(e)}")

```

```
return False
```

This adaptive approach allows the scraper to handle variations in HTML structure, such as tables with different class names or headers implemented in different ways.

3. Execution Process and Results

3.1 Web Page Fetching and Inspection

The scraping process begins with fetching the HTML content from the target URL and inspecting its structure:

```
=====
HOCKEY STANDINGS WEB SCRAPER
=====
A tool for extracting, analyzing, and visualizing hockey team statistics
By: Amidu Dabor
Date: 18 May 2025
=====

Initializing scraper for URL: https://www.scrapethissite.com/pages/forms/
Output directory: ./hockey_data

Fetching webpage...
Successfully fetched page from https://www.scrapethissite.com/pages/forms/
  Status code: 200
  Content size: 49.3 KB

Inspecting HTML structure...
=====
```

HTML CONTENT INSPECTION

=====

Content Length: 50439 characters

Page Title: Hockey Teams: Forms, Searching and Pagination | Scrape This Site | A public sandbox for learning web scraping

PAGE STRUCTURE SUMMARY

Tables: 1

Forms: 1

Links: 32

Images: 3

TABLES INFORMATION:

Table #1:

Class: table

ID: N/A

Rows: 26

Headers: 9

Header Texts: Team Name, Year, Wins, Losses, OT Losses, Win %, Goals For (GF), Goals Against (GA), + / -

=====

This inspection confirms that the page contains a single table with hockey standings data, providing the information needed to guide the subsequent extraction process.

3.2 Data Extraction and Inspection

After identifying the target table, the scraper extracts the structured data and performs a comprehensive inspection:

Extracting table data...

Successfully extracted 25 rows and 9 columns of data

Table headers: Team Name, Year, Wins, Losses, OT Losses, Win %, Goals For (GF), Goals Against (GA), + / -

Inspecting extracted data...

=====

EXTRACTED DATA INSPECTION

=====

Shape: (25, 10) (rows, columns)

Column Information:

- Team Name

Type: object

Non-Null: 25/25 (100.0%)

Unique Values: 21

- Year

Type: int64

Non-Null: 25/25 (100.0%)

Unique Values: 2

- Wins

Type: int64

Non-Null: 25/25 (100.0%)

Unique Values: 18

- Losses

Type: int64

Non-Null: 25/25 (100.0%)

Unique Values: 17

- OT Losses

Type: float64

Non-Null: 0/25 (0.0%)

Unique Values: 0

Missing Values: 25 (100.0%)

- Win %

Type: float64

Non-Null: 25/25 (100.0%)

Unique Values: 18

- Goals For (GF)

Type: int64
Non-Null: 25/25 (100.0%)
Unique Values: 23
- Goals Against (GA)
Type: int64
Non-Null: 25/25 (100.0%)
Unique Values: 23
- + / -
Type: object
Non-Null: 25/25 (100.0%)
Unique Values: 23
- Scraped_Date
Type: object
Non-Null: 25/25 (100.0%)
Unique Values: 1

First 10 rows:

	Team Name	Year	Wins	Losses	OT Losses	Win %	Goals For (GF)	Goals Against (GA)	+ / -	Scraped_Date	Win-Loss Ratio
0	Boston Bruins	1990	44	24	nan	0.55	299	264	35	2025-05-19 13:56:48	1.83333
1	Buffalo Sabres	1990	31	30	nan	0.388	292	278	14	2025-05-19 13:56:48	1.03333
2	Calgary Flames	1990	46	26	nan	0.575	344	263	81	2025-05-19 13:56:48	1.76923
3	Chicago Blackhawks	1990	49	23	nan	0.613	284	211	73	2025-05-19 13:56:48	2.13043
4	Detroit Red Wings	1990	34	38	nan	0.425	273	298	-25	2025-05-19 13:56:48	0.894737
5	Edmonton Oilers	1990	37	37	nan	0.463	272	272	0	2025-05-19 13:56:48	1
6	Hartford Whalers	1990	31	38	nan	0.388	238	276	-38	2025-05-19 13:56:48	0.815789
7	Los Angeles Kings	1990	46	24	nan	0.575	340	254	86	2025-05-19 13:56:48	1.91667
8	Minnesota North Stars	1990	27	39	nan	0.338	256	266	-10	2025-05-19 13:56:48	0.692308
9	Montreal Canadiens	1990	39	30	nan	0.487	273	249	24	2025-05-19 13:56:48	1.3

The inspection reveals detailed information about the extracted data, including:

- The structure of the dataset (25 rows, 10 columns)
- Data types for each column (automatically converted to appropriate types)
- Missing value counts (100% missing in "OT Losses" column)
- Unique value counts for each column A
- preview of the first 10 rows of data

This inspection helps identify potential data quality issues that need to be addressed during analysis.

3.3 Data Analysis

After extraction and inspection, the scraper performs comprehensive analysis of the hockey standings data:

```
Analyzing data...
Available columns: ['Team Name', 'Year', 'Wins', 'Losses', 'OT Losses', 'Win %', 'Goals For (GF)', 'Goals Against (GA)', '+ / -', 'Scraped_Date']
Data analysis completed with 10 metrics calculated

=====
HOCKEY DATA ANALYSIS RESULTS
=====

Total Teams: 25
Years Covered: 1990, 1991

Team Performance Metrics:
Average Wins: 34.48
Average Losses: 34.52
Average Win Percentage: 0.43%
Maximum Goals For: 344
Minimum Goals Against: 211
```

```
Team Highlights:
Best Win-Loss Ratio: St. Louis Blues (1990) with 2.14 (47 wins, 22 losses)
Most Wins: Chicago Blackhawks (1990) with 49 wins
Most Losses: Quebec Nordiques (1990) with 50 losses
```

The analysis reveals several key insights:

- The dataset covers 25 teams from two seasons (1990-1991)
- Teams averaged 34.48 wins and 34.52 losses per season
- The average win percentage was 43%
- The Chicago Blackhawks (1990) had the most wins (49)
- The St. Louis Blues (1990) had the best win-loss ratio (2.14) The
- Quebec Nordiques (1990) had the most losses (50)

3.4 Data Summary

The scraper also generates a concise summary of key statistics:

```
Creating data summary...
=====
HOCKEY DATA SUMMARY
=====
Total Teams: 25.00
Average Wins: 34.48
Max Wins: 49
Min Wins: 16
Average Losses: 34.52
```

```
Max Losses: 50
Min Losses: 22
Average Win %: 0.43
Max Win %: 0.61
Min Win %: 0.20
Average Goals For (GF): 276.68
Max Goals For (GF): 344
Min Goals For (GF): 223
Average Goals Against (GA): 276.80
Max Goals Against (GA): 354
Min Goals Against (GA): 211
```

This summary provides a quick overview of the distribution of key metrics, making it easy to understand the range and central tendency of important statistics.

3.5 Data Visualization

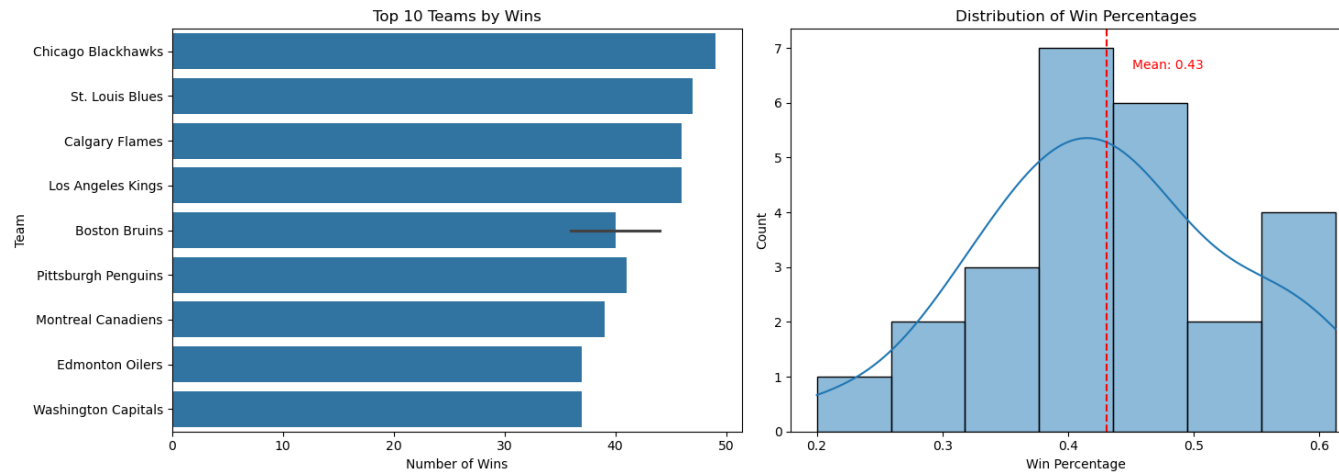
The scraper creates visual representations of key hockey statistics:

```
Creating visualizations...
Available columns for visualization: ['Team Name', 'Year', 'Wins', 'Losses', 'OT Losses', 'Win %', 'Goals For (GF)', 'Goals Against (GA)', '+ / -',
'Scraped_Date', 'Win-Loss Ratio']
Visualizations saved to: ./hockey_data/visualizations/hockey_viz_20250519_135648.png
Visualizations created successfully
```

The visualization includes two key plots:

1. A bar chart showing the top 10 teams by number of wins
2. A histogram showing the distribution of win percentages across all teams

Hockey Team Statistics Visualization



These visualizations provide clear visual insights into team performance and the distribution of win rates across the league.

3.6 Data Export

Finally, the scraper exports the processed data to a CSV file for further use:

```
Saving data to CSV...
Successfully saved data to ./hockey_data/Hockey_Data_20250519_135648.csv
  Rows: 25, Columns: 11
  File size: 2.45 KB

Verifying saved CSV file...
CSV FILE INSPECTION
=====
File Path: ./hockey_data/Hockey_Data_20250519_135648.csv
File Size: 2.45 KB
```

Last Modified: 2025-05-19 13:56:48

Number of Rows: 25

Number of Columns: 11

Column Names:

1. Team Name
2. Year
3. Wins
4. Losses
5. OT Losses
6. Win %
7. Goals For (GF)
8. Goals Against (GA)
9. + / -
10. Scraped_Date
11. Win-Loss Ratio

The exported CSV file contains all the extracted and processed data, including the original team statistics from the website and computed metrics like Win-Loss Ratio.

3.7 Complete Workflow Execution

The entire scraping workflow completes successfully with a summary of the execution:

WORKFLOW SUMMARY

=====

Status: Completed successfully

Steps completed: 9/9

Execution time: 2.28 seconds

Output files:

- CSV data: ./hockey_data/Hockey_Data_20250519_135648.csv

```
- Visualizations: ./hockey_data/visualizations
=====

Project Deliverables:
1. CSV file with extracted hockey data
2. Visualization of key hockey statistics
3. Data summary table
=====

Hockey data extraction workflow completed successfully in 2.28 seconds!
CSV data saved to: ./hockey_data/Hockey_Data_20250519_135648.csv.
```

The workflow successfully completed all 9 steps in just 2.28 seconds, producing both the CSV data file and visualizations as required by the project objectives.

4. Key Implementation Features

4.1 Automatic Data Type Conversion

The scraper automatically converts string data to appropriate numeric types for analysis:

```
def _convert_numeric_columns(self):
    """Convert appropriate columns to numeric data types."""
    # Hockey-specific numeric columns
    numeric_columns = ['Year', 'Wins', 'Losses', 'OT Losses', 'Win %',
                       'Goals For (GF)', 'Goals Against (GA)', '+/-']

    for col in numeric_columns:
        if col in self.data_frame.columns:
            # Convert to numeric with NaN handling
            self.data_frame[col] = pd.to_numeric(self.data_frame[col], errors='coerce')
```

This automatic conversion ensures that subsequent analysis and visualization functions work correctly with numeric data.

4.2 Retry Logic with Exponential Backoff

The implementation includes robust retry logic for network requests:

```
def fetch_page(self, max_retries=3, delay=2):
    """Fetch the webpage with retry logic."""
    for attempt in range(max_retries):
        try:
            response = requests.get(self.url, headers=self.headers, timeout=10)
            response.raise_for_status()

            # Process successful response
            # ...

            return True

        except requests.exceptions.RequestException as e:
            if attempt < max_retries - 1:
                # Implement exponential backoff
                wait_time = delay * (2 ** attempt)
                time.sleep(wait_time)
            else:
                return False
```

This approach ensures that temporary network issues don't cause the entire process to fail, with exponential backoff to avoid overwhelming the server.

4.3 Adaptive Table Extraction

The implementation can handle various table structures:

```
def extract_table_data(self, table_class='table'):
    """Extract data with multiple fallback strategies."""
    # Try finding table by class
    table = self.soup.find('table', class_=table_class)

    # Fallback to any table if class-based search fails
    if not table:
        table = self.soup.find('table')

    # Try different header extraction strategies
    header_elements = table.find_all('th')
    if header_elements:
        # Extract from th elements
        self.table_headers = [header.text.strip() for header in header_elements]
    else:
        # Try using first row as headers
        first_row = table.find('tr')
        self.table_headers = [cell.text.strip() for cell in first_row.find_all(['td', 'th'])]
```

This flexibility allows the scraper to work with different HTML structures, making it more robust and reusable.

4.4 Comprehensive Logging

The implementation includes detailed logging at multiple levels:

```
def _setup_logging(self, log_level):
    """Set up comprehensive logging configuration."""
    self.logger = logging.getLogger("HockeyScraper")
    self.logger.setLevel(log_level)

    # Create log directory
    log_dir = "./logs"
    os.makedirs(log_dir, exist_ok=True)

    # File handler with timestamp
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    log_file = os.path.join(log_dir, f"hockey_scraper_{timestamp}.log")
    file_handler = logging.FileHandler(log_file)

    # Console handler
    console_handler = logging.StreamHandler()

    # Create formatter
    formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelname)s - %(message)s')
    file_handler.setFormatter(formatter)
    console_handler.setFormatter(formatter)

    # Add handlers to logger
    self.logger.addHandler(file_handler)
    self.logger.addHandler(console_handler)
```

This logging system provides a detailed record of the scraping process, making it easier to debug issues and track performance.

5. Challenges and Solutions

Throughout the development process, I encountered several challenges that required creative solutions:

5.1 HTML Structure Variability

Challenge: The structure of HTML tables can vary significantly across websites, making it difficult to create a generic scraper.

Solution: Implemented an adaptive extraction strategy with multiple fallback mechanisms:

- Attempting to find tables by class name first
- Falling back to any available table if class-based search fails
- Supporting both dedicated header elements and first-row-as-headers approaches
- Adding comprehensive validation and normalization for row data

5.2 Data Type Conversion

Challenge: The extracted data often comes as strings, requiring conversion to appropriate numeric types for analysis.

Solution: Implemented automated type detection and conversion for common hockey statistics columns, using pandas' `to_numeric` function with appropriate error handling to ensure data integrity.

5.3 Visualization Adaptability

Challenge: Creating visualizations that would work regardless of the specific columns available in the data.

Solution: Developed a flexible visualization approach that:

- Automatically detects appropriate columns for visualization
- Provides meaningful fallback options when preferred columns are unavailable
- Adapts chart types based on available data (bar charts, histograms, or scatter plots)
- Includes proper axis labelling and formatting regardless of the visualization type

5.4 Error Handling and Resilience

Challenge: Web scraping is inherently prone to errors due to network issues, server responses, and changing page structures.

Solution: Implemented comprehensive error handling throughout the application:

- Retry mechanisms for network requests with exponential backoff
 - Detailed logging at various levels (INFO, WARNING, ERROR) for debugging
 - Graceful fallbacks when preferred extraction methods fail
 - Clear error messages and status reporting to the user
 - Step-specific error handling to prevent complete workflow failure
-

6. Conclusion

6.1 Project Achievements

This project successfully demonstrates the development of a robust, object-oriented web scraper for extracting hockey standings data. The implementation fulfills all the original objectives:

1. **Data Extraction:** Successfully extracts structured hockey team data from the target website

2. **Data Processing:** Cleans, validates, and normalizes the extracted data
3. **Data Analysis:** Performs comprehensive analysis of team performance metrics
4. **Data Visualization:** Creates meaningful visualizations of key statistics
5. **Data Export:** Saves processed data to CSV format for further use

6.2 Key Takeaways

Through this project, I gained valuable insights and skills:

1. **Web Scraping Techniques:** Learned practical approaches for extracting structured data from HTML content
2. **Robust Error Handling:** Developed strategies for handling network issues, parsing errors, and data inconsistencies
3. **Object-Oriented Design:** Applied OOP principles to create a modular, maintainable codebase
4. **Data Processing Pipeline:** Built an end-to-end data processing workflow from extraction to analysis
5. **Data Visualization:** Created effective visual representations of complex data

6.3 Future Improvements

While the current implementation meets all requirements, several potential improvements could enhance its functionality:

1. **Multi-page Scraping:** Extend the scraper to handle pagination and extract data from multiple pages
2. **Advanced Filtering:** Add capabilities to filter teams by various criteria (e.g., year, performance metrics)
3. **Comparative Analysis:** Implement features to compare teams across different seasons or metrics

4. **Interactive Visualizations:** Develop interactive dashboards using libraries like Plotly
5. **Scheduled Execution:** Set up automated scheduling for regular data updates
6. **Database Integration:** Implement storage in a database for historical tracking and more complex queries

6.4 Final Thoughts

This project demonstrates the power of web scraping for data collection and analysis. By combining HTTP requests, HTML parsing, and data processing techniques, we can extract valuable information from web sources and transform it into structured, analyzable data. The object-oriented approach ensures that the implementation is modular, extensible, and maintainable, making it a solid foundation for future web scraping projects.

7. References

1. Beautiful Soup Documentation: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
2. Requests Library: <https://requests.readthedocs.io/>
3. Pandas Documentation: <https://pandas.pydata.org/docs/>
4. Matplotlib and Seaborn Documentation: <https://matplotlib.org/>
5. Target Website: <https://www.scrapethissite.com/pages/forms/>